Unsupervised Anomaly Detection in Spacecraft Telemetry Using an LSTM Autoencoder: An Artifact-Audited Case Study on NASA SMAP Data

Haytham Hussen Hamdan · Saleh Khaled · Alwalid Alfaleet
King Fahd University of Petroleum & Minerals (KFUPM), Dhahran, Saudi Arabia
Research supervision: Muzammil Behzad, Ph.D. | Manuscript version: October 2026 (not peer reviewed)

Abstract

Automated anomaly detection is a central challenge in spacecraft health monitoring because high-volume multivariate telemetry can exhibit complex temporal dependencies, changing operating conditions, and infrequent abnormal events. This study documents and critically evaluates a reconstruction-based detection prototype using a long short-term memory (LSTM) autoencoder applied to telemetry from the NASA Soil Moisture Active Passive (SMAP) spacecraft benchmark. A saved neural-network artifact and a point-aligned prediction archive were examined to establish the implemented architecture and independently recalculate classification metrics for one annotated P-2 test sequence. Across 8,179 archived points, 1,033 true positives, 79 false positives, 192 false negatives, and 6,875 true negatives were obtained, corresponding to precision of 92.90%, recall of 84.33%, F1-score of 88.40%, and a 1.14% false-positive rate among normal points. An interactive dashboard illustrates the relationship between telemetry, reconstruction errors, alarm flags, and event chronology. The audit also identifies limitations in the retained experimental provenance, including absent training code and preprocessing artifacts, architecture inconsistencies between the saved model and its documentation, and evaluation on a selected channel. Accordingly, the findings support a proof-of-concept case study rather than an independently reproducible or operationally validated detection method.

Index Terms—Spacecraft telemetry; anomaly detection; recurrent neural networks; LSTM autoencoder; reconstruction error; SMAP; experimental reproducibility.

1. Introduction

Modern spacecraft generate continuous telemetry streams describing subsystem operating states, sensor measurements, and control context. Detecting departures from expected behavior is difficult when abnormal patterns are rare, temporally structured, or not expressible through simple static thresholds. The monitoring objective is not merely to identify unusual values; it is to identify potentially consequential deviations early enough to support engineering investigation while limiting distracting false alerts. Methods that learn normal sequence patterns offer one possible way to augment conventional rules and operator expertise [1].

Recurrent neural networks, particularly LSTM architectures, have been investigated for spacecraft telemetry monitoring and general multivariate anomaly detection [1], [2]. Reconstruction-based models learn a representation of nominal time-series windows and score later windows according to the difficulty of reconstructing them. This can reduce dependence on fault labels during model fitting, although effective deployment still requires suitable calibration and careful evaluation.

This paper examines an LSTM-autoencoder prototype known as SatGuard AI using archived NASA SMAP benchmark outputs. Rather than claiming a novel neural architecture or state-of-the-art accuracy, the objective is to document an end-to-end system and distinguish results that can be reproduced directly from its retained artifacts from methodological details available only in development notes. The study contributes: (i) an auditable pointwise evaluation on an annotated SMAP sequence; (ii) an implementation-level characterization of the saved recurrent model and alerting logic; and (iii) a transparent assessment of validity threats and steps required for a publishable, independently replicable benchmark.

2. Related Work and Research Context

Hundman et al. [1] investigated LSTM-based modeling and dynamic thresholding using SMAP spacecraft and Mars Science Laboratory rover telemetry, demonstrating the importance of handling structured temporal errors and mitigating false positives. Their approach focuses on predictions and nonparametric thresholding and is therefore related to, but not identical to, the reconstruction-based approach assessed here. Malhotra et al. [2] studied LSTM encoder–decoder architectures for multivariate sensor anomaly detection and demonstrated how learned

reconstruction behavior can function as an anomaly score. The approach in the present project follows this general encoder–decoder rationale.

Subsequent research explored richer temporal and probabilistic representations. OmniAnomaly [3] uses stochastic recurrent modeling for multivariate signals, while the Anomaly Transformer [4] investigates deviations in learned temporal associations. These works provide relevant methodological context and future baselines, but they were not executed on an identical split in this study. Consequently, no superiority claim relative to these methods is warranted. Similarly, the selected telemetry archive cannot establish transferability to other spacecraft, operating regimes, or anomaly categories.

3. Dataset, Artifacts, and Evaluation Design

3.1. Source data and selected evaluation sequence

The project used the NASA/JPL spacecraft-anomaly dataset introduced in [1], obtained during development from a public SMAP/MSL dataset mirror [5]. The reported input represents a selected SMAP channel identified as P-2, accompanied by additional contextual variables. The archived network input uses 25 features at each time step. The surviving prediction file contains N = 8,179 test positions, an annotated anomaly spanning indices [5,320, 6,545) (1,225 positions), an error score for each position, and a binary detection indicator. Index positions rather than absolute UTC timestamps are used throughout because the retained file does not preserve the original timestamp mapping.

The evaluation denominator is therefore 1,225 labeled anomalous points plus 6,954 nominal points. This is a single selected sequence and a pointwise study; observations are temporally correlated, so metric percentages must not be interpreted as independent Bernoulli trials. The archive supports a retrospective detection assessment, but it is not sufficient to reconstruct acquisition, normalization, training, or inference timing end to end.

3.2. Artifact provenance and what was verified

The supplied project archive includes a serialized Keras neural network (satellite_lstm_model.keras), a Flask application and web templates, results.json and results_v2.json, package requirements, and internal project documentation. The metric audit in this manuscript uses results_v2.json, not numbers copied from the dashboard. Model structure was established from the serialized network artifact. Raw training arrays, the scaler fitted during training, the original training script/notebook, experiment seeds, and train/validation split indices were not present in the supplied archive. These absences are material to repeatability and preclude an independent training replication.

4. Anomaly-Detection Method

4.1. Reconstruction objective

Let $X_t \in \mathbb{R}^{w \times d}$ denote a multivariate telemetry window, with window length w and feature count d. An encoder E_θ maps X_t to a latent representation z_t; a decoder D_φ estimates the original window as $\hat{X}_t = D_\varphi(E_\theta(X_t))$. Learning on nominal data seeks parameters that minimize average reconstruction discrepancy. According to the development documentation, the training objective was mean squared error, $L_{MSE} = \frac{1}{w\,d} \sum_{i=1}^{w} \sum_{j=1}^{d} (X_{i,j} - \hat{X}_{i,j})^2$. The reported anomaly score at inference used a mean absolute reconstruction-error formulation. These are distinct quantities, and their exact pointwise aggregation cannot be derived from the saved model alone.

4.2. Model structure and reported training configuration

Inspection of the serialized model shows an input of 30 time steps × 25 features, an LSTM encoder with 64 units, a RepeatVector of length 30, an LSTM decoder with 64 units, and a time-distributed dense reconstruction head with 25 outputs. The saved weight shapes imply 57,689 trainable parameters. This architecture is a fact of the submitted model artifact, whereas the accompanying README describes 50-step windows and approximately 16,569 parameters. A recovered training notebook or model-conversion history would be needed to explain this discrepancy.

Component	Saved artifact (verified)	Documentation (unverified)
Input	30 × 25 time-series window	50 × 25 window
Encoder	LSTM, 64 units	LSTM, 64 units
Decoder	RepeatVector(30) → LSTM(64)	RepeatVector(50) → LSTM(64)
Output	TimeDistributed(Dense(25))	25-feature reconstruction
Trainable parameters	57,689	Approximately 16,569
Training settings	Not recoverable from weights alone	Adam; MSE; 50 epochs; batch 32

Table 1. Discrepancy between the inspectable Keras artifact and written development notes. “Verified” refers to serialized model structure, not re-executed training.

4.3. Preprocessing and alert decision

The development notes describe fitting a StandardScaler on normal training data, generating overlapping windows, and suppressing short score excursions. They also report 3σ outlier handling and a minimum run of approximately ten consecutive high-error positions. A fixed threshold is reported as $\tau = \mu_{train} + 3\sigma_{train}$, where μ_{train} and σ_{train} are the mean and standard deviation of reconstruction errors computed on nominal training data. The stored value for the final sequence is $\tau = 0.488742$. Because the scaler, error-aggregation code, and alert post-processing script are absent, the exact thresholds and archived flags cannot be regenerated independently from raw telemetry using the supplied files.

For the audit, a point is counted as a predicted anomaly when the stored binary flag equals 1. The reference annotation defines the ground-truth class: $y_t = 1$ for $5{,}320 \le t < 6{,}545$ and $y_t = 0$ otherwise. This choice avoids retroactively altering the labels or flags to improve metrics and permits a deterministic recount.

4.4. Evaluation measures

Given true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN), pointwise precision is TP/(TP + FP), recall is TP/(TP + FN), and F1 = 2·precision·recall/(precision + recall). The conventional false-positive rate is FP/(FP + TN), with a denominator restricted to normal points. This should not be confused with FP/N, which counts false alarms as a fraction of the complete dataset. Metrics are reported to two decimal places. The available archive does not support reliable eventwise latency or deployment-level alert-rate claims.

5. Results

5.1. Archived prediction audit

Recounting all 8,179 stored flags gives the confusion counts and derived measures in Table 2. Precision of 92.90% indicates that most archived positive flags fall inside the annotated interval; recall of 84.33% indicates that 192 labeled anomalous positions were not flagged. The dataset contains one contiguous labeled anomaly interval, so pointwise precision and recall alone do not establish event-level detection generalization. All 79 false-positive points occur before the annotated interval; none occur after it in this sequence. This is a property of the selected archived run, not a universal operating characteristic.

Measure	Value	Interpretation
Test sequence size	8,179	All archived positions
Ground-truth anomaly / normal	1,225 / 6,954	Reference interval / outside interval
True positives / false negatives	1,033 / 192	Detected / missed anomalous points
False positives / true negatives	79 / 6,875	Incorrect flags / correctly unflagged normal points
Precision / recall	92.90% / 84.33%	Pointwise positive predictive value / sensitivity
F1 score	88.40%	Harmonic mean of precision and recall
False-positive rate	1.14%	79 / 6,954 normal points
False alarms / all points	0.97%	79 / 8,179; not conventional FPR

Table 2. Independently recomputed pointwise results from results_v2.json. No model retraining or independent inference was performed.

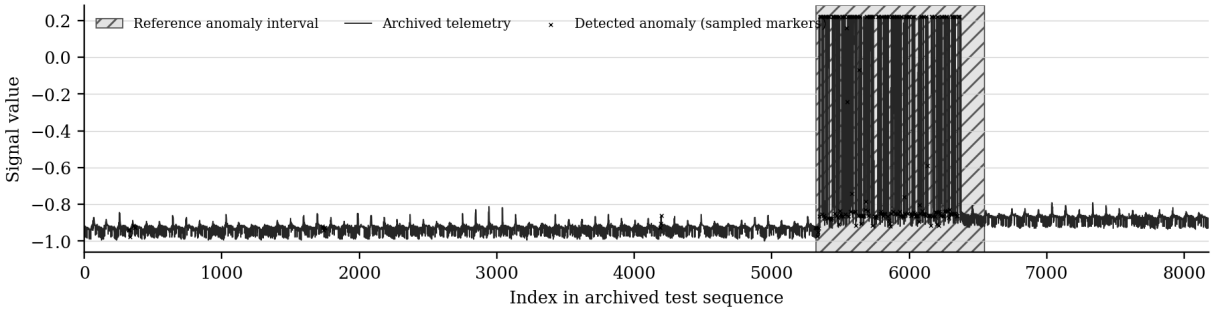


Figure 1. Archived P-2 telemetry and anomaly flags. The hatched region denotes the reference anomaly interval; cross markers are a visual subsample of positive flags to reduce overplotting. Horizontal positions are sample indices, not authenticated timestamps.

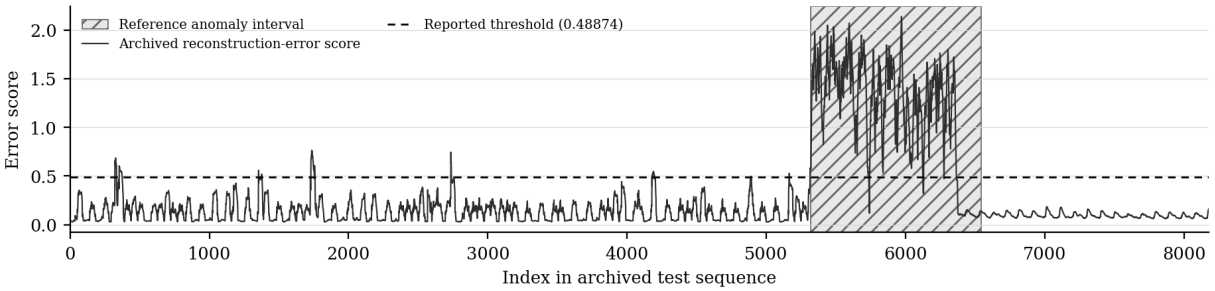


Figure 2. Archived reconstruction-error series and its reported fixed threshold. The plot depicts exported scores; it is not a new inference run.

5.2. Development iterations and comparison validity

Internal development notes list several exploratory models and channels, including single-feature and multivariate Isolation Forest approaches, a dense autoencoder, and recurrent-autoencoder variants. Their reported precision scores were 4.2%, 45.6%, 15.1%, 78.6%, and 92.9%, respectively. However, most iterations used different telemetry channels. These records are useful as an engineering history but do not constitute a controlled ablation study or same-dataset baseline comparison. They are not used to infer that the final architecture is superior to the alternatives.

5.3. Monitoring interface

The prototype provides a Flask-based application layer and a browser interface that displays telemetry traces, reconstruction-error values, model flags, alert summaries, and a replayable mission timeline. This separation between precomputed results and human-readable operational displays is valuable for demonstrating the workflow of an anomaly-monitoring system. However, the interface replays an archived result series; it does not ingest live SMAP telemetry, prove online inference throughput, or demonstrate autonomous on-orbit fault diagnosis. These capabilities should be evaluated separately before making real-time deployment claims.

6. Discussion and Threats to Validity

The observed performance indicates substantial temporal overlap between archived anomaly flags and one reference interval. Yet the experimental context imposes several constraints on scientific interpretation. First, selection of P-2 after reviewing multiple channels or their labels can bias estimates upward. Second, anomaly samples are concentrated in one long episode; neighboring points are correlated and metrics cannot be interpreted as 1,225 independent detection events. Third, the threshold calculation, persistence rule, and preprocessing sequence are not recoverable in executable form, so it is not possible to determine how much each component contributed to precision or recall.

Fourth, the saved architecture differs from project documentation, leaving unclear whether the archived weights are associated with the exact reported prediction file. Fifth, model calibration under class imbalance, seasonal drift, unseen fault classes, and mission-specific telemetry semantics has not been tested. The P-2 identifier alone cannot establish that the anomaly was physically caused by a specific relay or subsystem; the development notes' proposed hardware explanation is a hypothesis, not a validated diagnosis. Finally, there is no documented ground-station integration, resource-use measurement, or measured inference latency. These limitations do not negate the prototype but restrict the claims it can substantiate.

7. Reproducibility Requirements and Proposed Research Extensions

A subsequent controlled experimental study should (a) acquire the original SMAP/MSL data with documented checksums and licenses; (b) preserve train/validation/test boundaries and anomaly label mappings; (c) publish the preprocessing, scaler, sliding-window code, model configuration, random seeds, and hyperparameter selection protocol; (d) prove that threshold calibration uses no test labels; and (e) evaluate inference on untouched test channels with consistent windows and event definitions. Metrics should include eventwise recall, pointwise precision–recall curves, false alarms per unit time, detection delay, runtime, and memory footprint. Cross-channel and cross-mission testing, robust traditional baselines, LSTM encoder–decoder alternatives [2], and modern probabilistic or attention-based approaches [3], [4] would make generalization claims meaningful.

Further study could investigate calibration under mission regime changes, confidence-aware human review, interpretability of context features, and anomaly localization rather than only binary classification. From a systems standpoint, a streaming path would require reliable ingestion, backpressure handling, telemetry quality checks, versioned model and scaler artifacts, auditable alert histories, and explicit human-in-the-loop escalation. These are prospective directions, not capabilities established by the current archive.

8. Conclusion

This study documents a reconstruction-based spacecraft telemetry anomaly-detection prototype and critically audits its retained artifacts. The saved LSTM autoencoder and 8,179-point P-2 prediction archive support a deterministic report of 92.90% precision, 84.33% recall, 88.40% F1, and 1.14% normal-point false-positive rate for the selected sequence. The associated dashboard offers an interpretable demonstration of monitoring outputs. The scientific contribution at this stage is an honest, bounded case study and artifact-level audit. Reconstructing the full training pipeline and conducting multi-channel, label-blind evaluation are prerequisites for broader empirical conclusions.

Acknowledgments

The authors acknowledge Dr. Muzammil Behzad for research supervision. The original prototype received first place at SPACETHON 2026, organized with support from the KFUPM Aerospace Engineering Club and the Saudi Space Community. Authorship and order should be approved by all named contributors prior to public posting. The research descriptions and numbers in this version were reviewed against artifacts available as of 28 October 2026.

References

[1] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding,” in Proc. ACM SIGKDD, 2018, pp. 387–395. doi:10.1145/3219819.3219845.

[2] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection,” arXiv:1607.00148, 2016. https://arxiv.org/abs/1607.00148.

[3] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network,” in Proc. ACM SIGKDD, 2019, pp. 2828–2837. doi:10.1145/3292500.3330672.

[4] J. Xu, H. Wu, J. Wang, and M. Long, “Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy,” in Proc. ICLR, 2022. https://openreview.net/forum?id=LzQQ89U1qm_.

[5] P. Fleith, “NASA Anomaly Detection Dataset – SMAP & MSL,” Kaggle data mirror cited in project documentation. https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl.

[6] H. H. Hamdan, S. Khaled, and A. Alfaleet, “SatGuard AI,” internal project archive (README, serialized model, web code, prediction exports, and technical report), April–May 2026. Unpublished project materials.

Appendix A. Artifact Audit and Reporting Scope

Audited files: satellite_lstm_model.keras (serialized network topology and weights), results_v2.json (8,179-element signal, score and flag vectors, reference interval, reported threshold), app.py (dashboard delivery layer), templates/index.html (interactive presentation), and the accompanying README/PDF documentation. All numerical results and figures are derived from archived outputs. Reproduction of the training process, re-extraction of original NASA data, and confirmation of spacecraft subsystem identity were outside the scope of this audit. Prior to submission to any venue, the author list, affiliations, accuracy of methodological provenance, and any public-release permissions should be confirmed by the collaborators.